République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Département d'Intelligence Artificielle et Sciences des Données (DIASD)

Master 1 Systèmes Informatiques Intelligents

Mini Project Agent Technologies

Multi-Agent Systems

Presented by:
Moussaoui Sarah
Fekir Chehla Nabila
Groupe 3

Academic Year 2024/2025

Table des Matières

Liste des Figures

Listings

0.1 Introduction

Artificial Intelligence increasingly relies on Multi-Agent Systems (MAS) to model complex behaviors and interactions among autonomous entities. This mini-project focuses on two essential aspects of MAS: multi-agent negotiation and the use of mobile agents in environments that require multi-criteria decision-making.

To begin with, the project explores the Partial Order Planner from the AIMA (Artificial Intelligence: A Modern Approach) repository. This offers valuable hands-on experience with classical AI planning algorithms, grounding the theoretical background before diving into implementation-specific challenges.

The first core component of the project simulates a competitive auction scenario involving a single seller and multiple buyers. Here, buyers incrementally raise their bids, while the seller selects an offer based on a concealed reserve price. This simulation provides insight into MAS applications in fields such as automated negotiation and e-commerce, emphasizing strategic interaction in dynamic markets.

The second part introduces the concept of mobile agents, where a buyer agent migrates across different containers or platforms to interact with multiple sellers. This illustrates the flexibility and efficiency of mobile agents in distributed environments, especially when decisions must consider multiple evaluation criteria.

Altogether, this project aims to deepen our understanding of agent behavior, negotiation dynamics, mobility, and distributed decision-making within MAS, while bridging foundational AI concepts with real-world implementations.

0.2 Multi-Agent Negotiation (One Seller – Multiple Buyers)

This section presents the implementation of a multi-agent auction system, featuring a single seller agent and multiple buyer agents engaged in competitive bidding. The simulation mirrors a real-world auction, where buyers progressively increase their offers, and the seller selects the winning bid based on the highest proposal that surpasses a concealed reserve price.

0.2.1 Auction Protocol Description

The multi-agent auction protocol is composed of the following stages:

1. **Auction Initialization:** The `SellerAgent` announces an initial offer (e.g., 100 units) to all participating buyer agents.
2. **Dynamic Bidding:** Buyers reactively submit incremented offers based on the current highest bid.
3. **Broadcast Mechanism:** After each bidding round, the seller notifies all agents of the latest top offer.
4. **Termination Conditions:** The auction concludes if:
   - No bids are received for a continuous 30-second period.
   - All buyer agents hit their budget thresholds.
5. **Winner Selection:** The final bid is validated against a confidential reserve price. If the condition is satisfied, the winning buyer is declared.

0.2.2 Design of `SellerAgent`

Functionality: The `SellerAgent` orchestrates the auction, managing price dissemination, bid evaluation, and winner selection.

Agent Behaviours:

- `SendStartingPriceBehaviour` (*OneShotBehaviour*): Broadcasts the initial asking price to all buyers at auction start.
- `ReceiveBidsBehaviour` (*CyclicBehaviour*):
  - Continuously listens for incoming offers.
  - Updates the highest bid value.
  - Broadcasts updates to all buyers.
  - Triggers auction termination after 30 seconds of inactivity.

Key Control Logic:

- Accepts only bids that strictly exceed the current best offer.
- Verifies whether the final accepted bid surpasses a predefined hidden reserve price.
- Declares a winner if the reserve price condition is met.

0.2.3 Implementation of `BuyerAgent`

Objective: The `BuyerAgent` competes in the auction by evaluating the current offer and submitting a higher bid, within its budget constraints.

Behaviours:

- `ReceiveOfferBehaviour` (*CyclicBehaviour*):
  - Monitors new bid announcements from the seller.
  - Responds by submitting a bid incremented by a random value between 0 and 10 units.
  - Withdraws from bidding once its budget (ranging between 1000 and 1500 units) is exceeded.
- `StopBiddingBehaviour` (*CyclicBehaviour*):
  - Transitions the agent to a passive state.
  - Continues receiving auction messages without active participation.

Decision-Making Mechanism:

- Operates autonomously using a reactive model.
- Generates stochastic bid increments within defined limits.
- Enforces strict adherence to local budget constraints.

0.2.4 System Output and Visualization

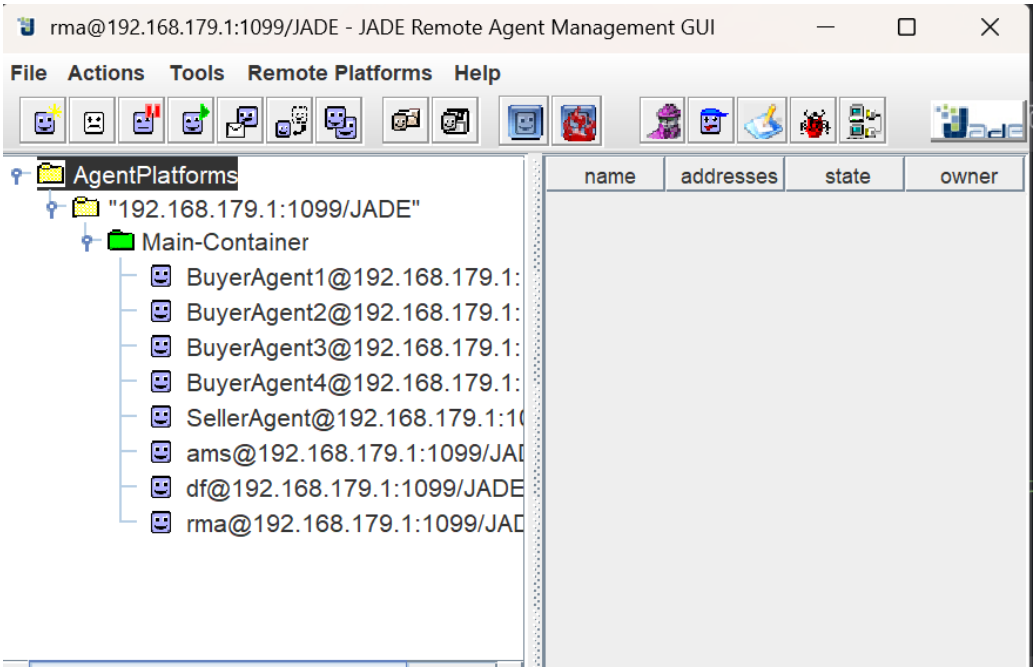

Figure 1: Agent Initialization in JADE Environment

System Log: The simulation confirms the successful deployment of one `SellerAgent` and four `BuyerAgents`, each registered within the JADE main container.

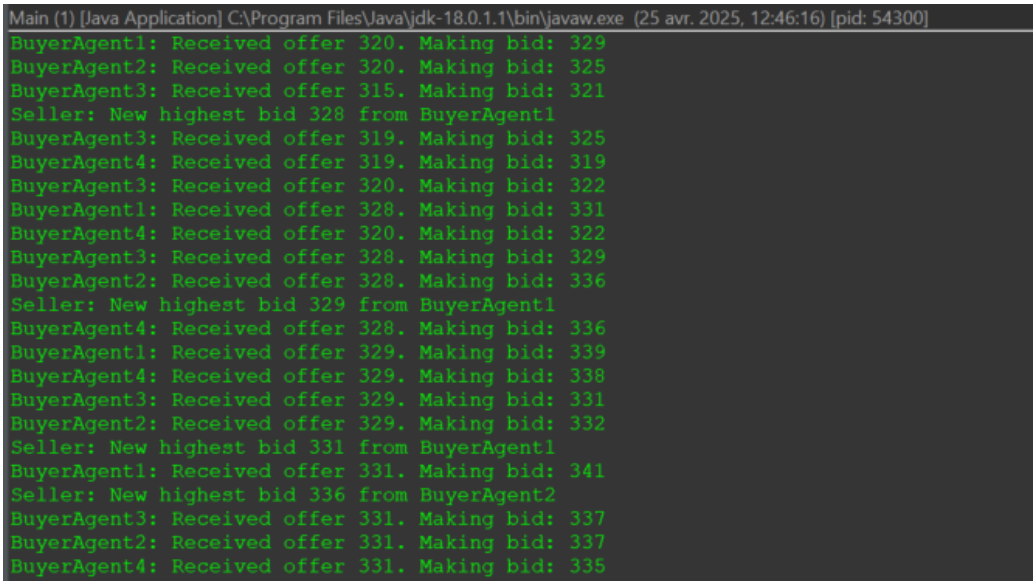

Figure 2: Active Bidding Round – Stage 1

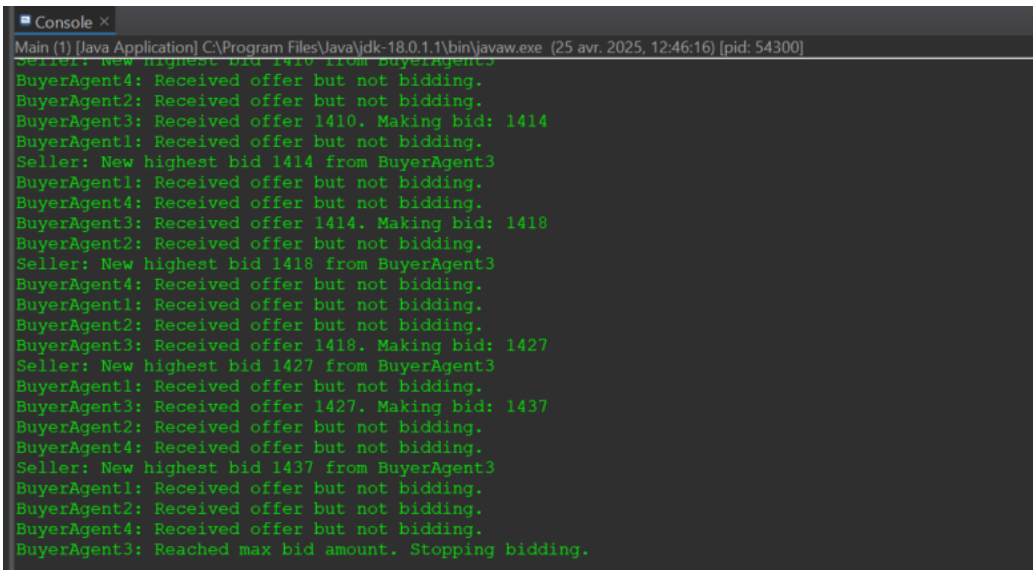

Figure 3: Auction Progression with Bid Updates

0.3 Mobile Agent-Based Multi-Criteria Decision Making (1 Buyer – Multiple Sellers)

This module extends the multi-agent architecture by integrating two advanced paradigms: mobile agents and multi-criteria decision analysis (MCDA). In this scenario, a single autonomous buyer agent is designed to interact with several seller agents distributed across containers. The buyer evaluates offers based on a weighted set of criteria, leveraging mobility to navigate the distributed system.

0.3.1 Agent Mobility and Distributed Environment

Mobility in agent-based systems introduces the capability for agents to transition between distinct execution contexts. In this configuration, inter-container mobility enables the `BuyerAgent` to traverse multiple containers hosted within the same JADE platform.

System Configuration:

- **Agent Roles:**
  - `BuyerAgent` — mobile, decision-making agent.
  - `SellerAgents` (4 instances) — offer providers.
- **Container Topology:**
  - `Main-Container` — platform core and initialization hub.
  - `Seller-Container` — hosts all seller agents.
  - `Buyer-Container` — initial deployment site for the buyer agent.

0.3.2 Operational Protocol

Interaction Flow:

1. The buyer broadcasts a `Call for Proposals (CFP)` to all seller agents.
2. Each seller responds with a proposal including multiple criteria (e.g., price, delivery time, and quality).
3. Proposals are evaluated using a dedicated utility function provided by a `ProductEvaluator` module.
4. The optimal offer is selected based on predefined weights assigned to each criterion.

Evaluation Logic:

- Multi-criteria utility functions are employed to rank seller responses.
- Criteria are weighted according to buyer preference profiles.
- Decision-making is fully autonomous and embedded in the buyer agent.

0.3.3 Mobility Features and Inter-Agent Communication

Agent Migration:

- `doMove()` is explicitly invoked for container traversal.
- Migration enables decentralized data acquisition across container boundaries.

Communication Layer:

- Agents utilize JADE's ACL message-passing for inter-container dialogue.
- The infrastructure is architected for scalability, with future support for inter-platform migration.

Program output

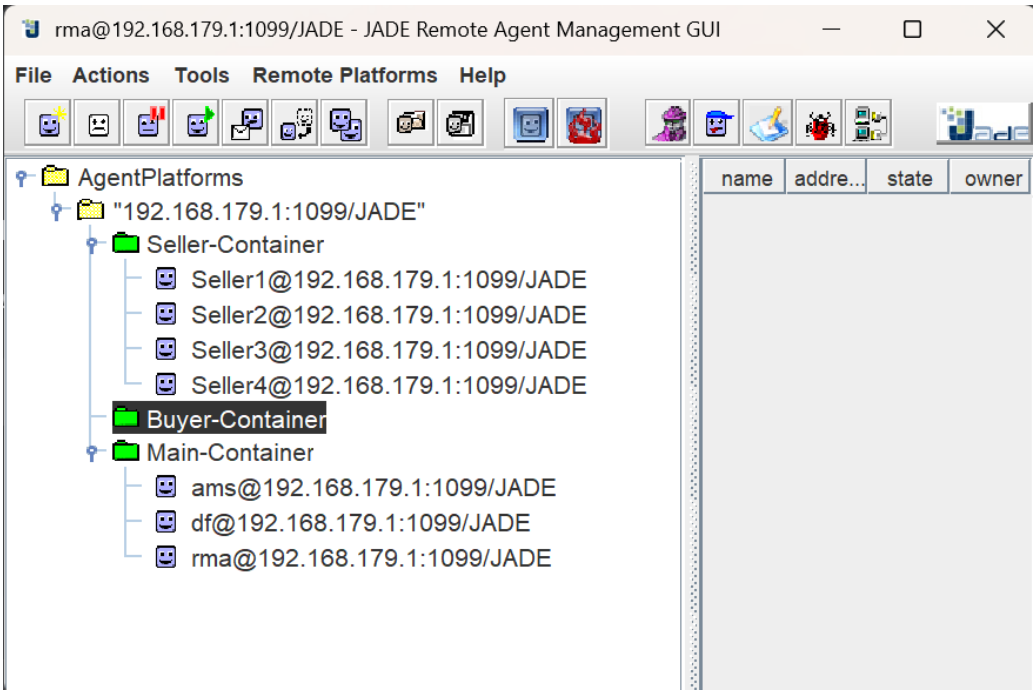

Figure 4: jade 2 intercontainer

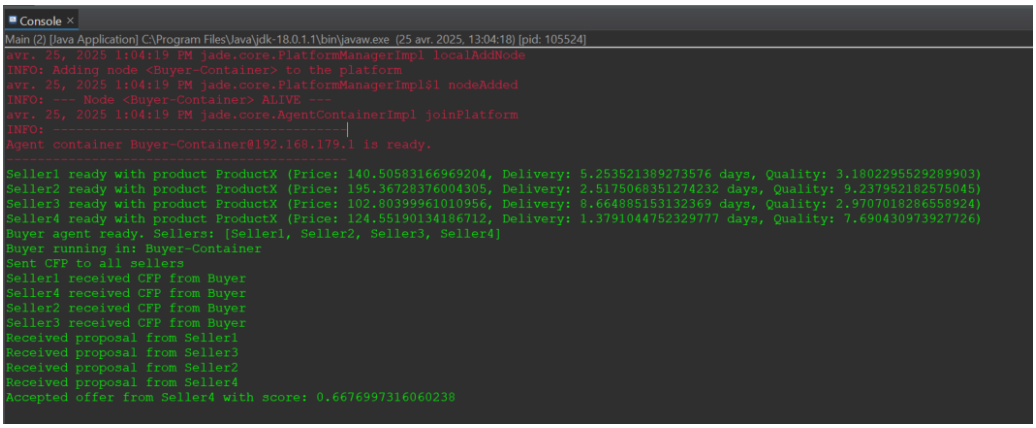

Figure 5: terminal execution intercontainer

0.3.4 Inter-Platform Migration

Inter-platform migration represents a sophisticated extension of agent mobility within multi-agent systems. Unlike inter-container migration, which occurs within a single platform, this mechanism enables a mobile agent to relocate between multiple independent platforms—often distributed across different machines or network locations.

In this scenario, the buyer agent migrates from its originating platform to remote seller platforms to directly engage with seller agents. This capability reduces communication overhead, grants access to localized resources or data, and increases the agent's operational autonomy in a distributed environment.

Migration Workflow

1. Initialization of the buyer platform via `BuyerPlatformMain`.
2. Initialization of the seller platform via `SellerPlatformMain`, hosting four seller agents.

3. Creation of the buyer agent on the buyer platform.
4. Buyer agent initiates migration to the seller platform.
5. Upon successful migration, the buyer requests proposals from seller agents.
6. The buyer evaluates received offers, selects the optimal one, and proceeds with order placement.

Platform Setup

The platforms are configured with mobility services enabled and designated container names to support agent migration:

- **BuyerPlatformMain.java**
  - Container identifier: `"Main-Container-buyer"`
  - Mobility services activated
- **SellerPlatformMain.java**
  - Container identifier: `"Main-Container-seller"`
  - Mobility services activated

Migration Implementation in `BuyerAgent.java`

The buyer agent incorporates two principal behaviors to manage migration:

- **Migration Behavior** (`CyclicBehaviour`):
  - Attempts to migrate to `"Main-Container-seller"` up to three times.
  - Executes migration via the `doMove(destination)` method.
  - Waits a 3-second interval between successive attempts.
- **Location Monitoring Behavior** (`CyclicBehaviour`):
  - Periodically checks the current container location using `here()`.
  - Upon confirming arrival at the target platform, initiates the `RequestOffers` behavior.
  - Terminates itself after successful migration.

method handling the migration

```
protected void setup() {
    System.out.println("Agent " + getLocalName() + " started.");

    addBehaviour(new OneShotBehaviour() {
        @Override
        public void action() {
            try {
                ContainerID destination = new ContainerID();
                destination.setAddress("localhost");
                destination.setPort("1098");

                doMove(destination);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}
```

Listing 1: Agent Migration Setup in `setup()` Method

Program output

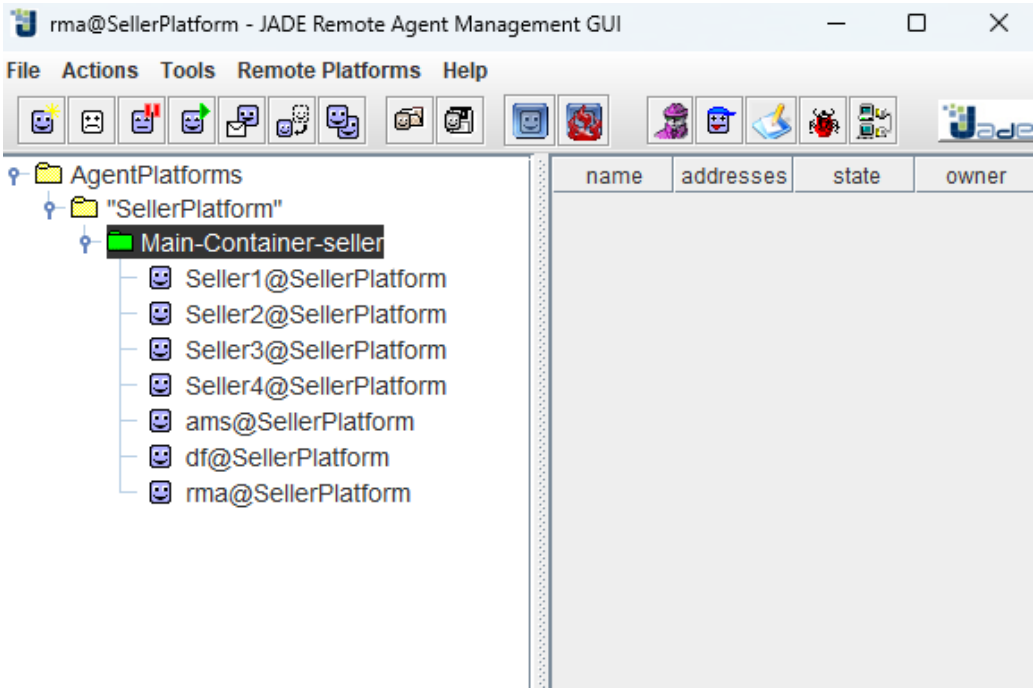

Figure 6: platform 1 sellers

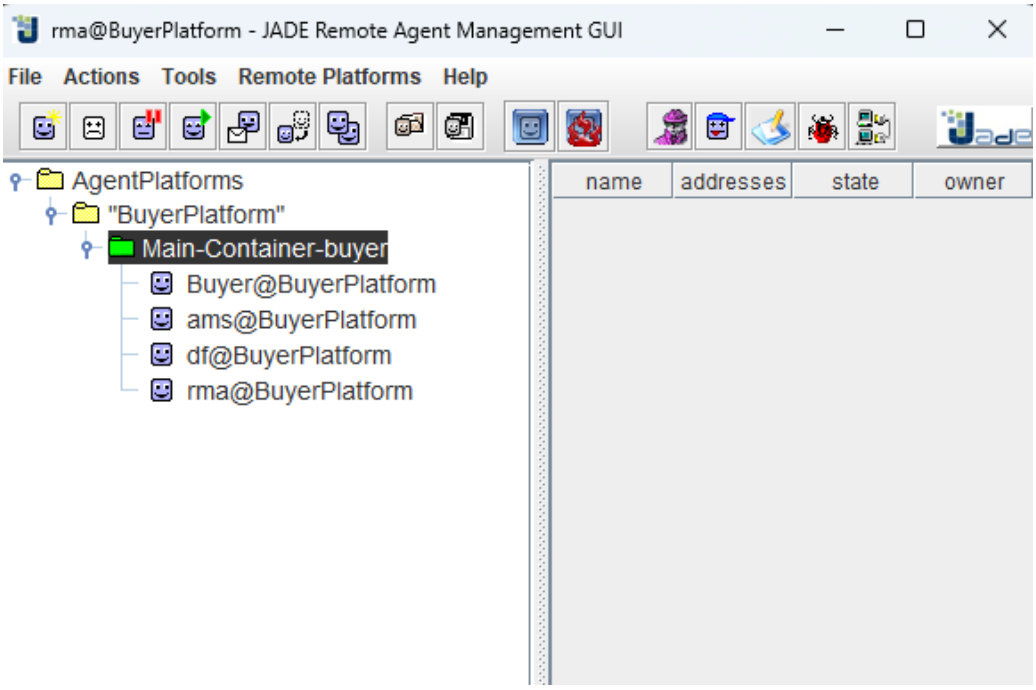

Figure 7: platform 2 buyer

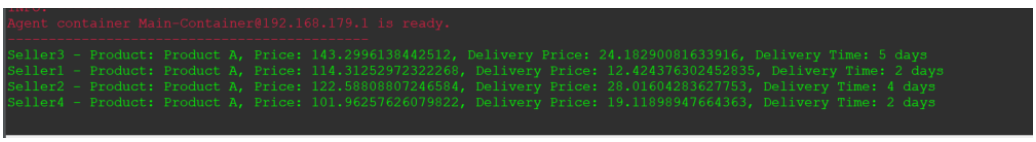

Figure 8: Running code of the seller first

```
BuyerAgent current location: Main-Container-buyer
Max migration attempts reached. Giving up.
BuyerAgent current location: Main-Container-buyer
BuyerAgent current location: Main-Container-buyer
BuyerAgent current location: Main-Container-buyer
BuyerAgent current location: Main-Container-buyer
BuyerAgent current location: Main-Container-buyer
BuyerAgent current location: Main-Container-buyer
```

Figure 9: Running code of the buyer

comment: It can be observed that the Buyer agent remains in its original container and does not perform inter-platform migration to the container hosting the Seller agents, as expected.

0.4 Evaluation of the AIMA Partial Order Planning Code

0.4.1 Project Overview: Partial Order Planner Implementation in AIMA-Python

Planning is a cornerstone task within **Artificial Intelligence (AI)**, where the objective is to devise a sequence of actions that transition an initial state to a desired goal. Among various planning techniques, the **Partial Order Planner (POP)**, as presented in the seminal textbook *Artificial Intelligence: A Modern Approach* (AIMA), offers a powerful strategy that avoids committing to a fixed action sequence prematurely. Instead, it builds plans by maintaining only necessary ordering constraints between actions.

This project explores the design and testing of a POP algorithm implemented in the `AIMA-Python` repository. The planner can tackle classical AI problems such as the *blocks world* and *logistics* domains, along with customized scenarios defined by the user. Unlike conventional planners that enforce a strict linear order of steps, POP operates with **causal links** and **partial ordering**, enabling more flexible scheduling and efficient management of action dependencies.

Aims of the Project

- Gain a comprehensive **understanding** of AI planning concepts, including:
  - The role of action preconditions and effects in planning
  - The formation and maintenance of causal links, and techniques for resolving conflicts (threats)
  - The contrast between partial order and total order planning frameworks
- **Implement** the POP algorithm and apply it to standard benchmark problems such as the $\texttt{spare}_t ire_p roblemand$

Relevant Use Cases

- Motion and task planning in robotics
- Automation of industrial workflows and business processes
- Behavior planning for non-player characters (NPCs) in video games

This project is intended for AI practitioners and learners focused on mastering symbolic planning techniques, emphasizing clear algorithms and educational clarity through the AIMA codebase.

Output Examples and Interpretation

```
Causal Links
(PutOn(Spare, Axle), At(Spare, Axle), Finish)
(Start, Tire(Spare), PutOn(Spare, Axle))
(Remove(Flat, Axle), NotAt(Flat, Axle), PutOn(Spare, Axle))
(Start, Tire(Flat), Remove(Flat, Axle))
(Start, At(Flat, Axle), Remove(Flat, Axle))
(Remove(Spare, Trunk), At(Spare, Ground), PutOn(Spare, Axle))
(Start, Tire(Spare), Remove(Spare, Trunk))
(Start, At(Spare, Trunk), Remove(Spare, Trunk))
(Remove(Flat, Axle), At(Flat, Ground), Finish)

Constraints
Start < PutOn(Spare, Axle)
Start < Finish
PutOn(Spare, Axle) < Finish
Remove(Flat, Axle) < PutOn(Spare, Axle)
Start < Remove(Spare, Trunk)
Start < Remove(Flat, Axle)
Remove(Flat, Axle) < Finish
Remove(Spare, Trunk) < PutOn(Spare, Axle)

Partial Order Plan
[{Start}, {Remove(Flat, Axle), Remove(Spare, Trunk)}, {PutOn(Spare, Axle)}, {Finish}]
```

Figure 10: Example Partial Order Plan 1

Analysis: In this plan, the actions `Remove(Flat, Axle)` and `Remove(Spare, Trunk)` are grouped together, indicating that their relative execution order does not affect the plan outcome. However, the `PutOn(Spare, Axle)` action must follow completion of both removal steps, reflecting the essential causal relationships inherent to the task.

```
Causal Links
(FromTable(B, A), On(B, A), Finish)
(FromTable(C, B), On(C, B), Finish)
(Start, Clear(C), FromTable(C, B))
(Start, OnTable(C), FromTable(C, B))
(Start, OnTable(B), FromTable(B, A))
(Start, Clear(A), FromTable(B, A))
(ToTable(A, B), Clear(B), FromTable(C, B))
(Start, On(A, B), ToTable(A, B))
(Start, Clear(A), ToTable(A, B))
(ToTable(A, B), Clear(B), FromTable(B, A))

Constraints
Start < FromTable(C, B)
FromTable(B, A) < Finish
Start < Finish
ToTable(A, B) < FromTable(B, A)
Start < ToTable(A, B)
ToTable(A, B) < FromTable(C, B)
Start < FromTable(B, A)
FromTable(B, A) < FromTable(C, B)
FromTable(C, B) < Finish

Partial Order Plan
[{Start}, {ToTable(A, B)}, {FromTable(B, A)}, {FromTable(C, B)}, {Finish}]
```

Figure 11: Example Partial Order Plan 2

Analysis: This particular plan is strictly sequential with no flexibility in action ordering; each step must be executed in the presented sequence to successfully reach the target state.

```
Causal Links
(LeftShoe, LeftShoeOn, Finish)
(LeftSock, LeftSockOn, LeftShoe)
(RightShoe, RightShoeOn, Finish)
(RightSock, RightSockOn, RightShoe)

Constraints
RightShoe < Finish
LeftSock < LeftShoe
Start < RightSock
LeftShoe < Finish
RightSock < RightShoe
Start < RightShoe
Start < Finish
Start < LeftSock
Start < LeftShoe

Partial Order Plan
[{Start}, {LeftSock, RightSock}, {RightShoe, LeftShoe}, {Finish}]
```

Figure 12: Example Partial Order Plan 3

Analysis: The plan allows some flexibility regarding when socks and shoes are worn, as long as both socks are put on before any shoes. However, the planner did not identify the valid total-order solution sequence `LeftSock → LeftShoe → RightSock → RightShoe`, because such specific sequences fall outside the scope of general partial-order plans and represent total-order constraints instead.

0.5 Conclusion

This mini-project offered a practical and comprehensive exploration of fundamental concepts in Multi-Agent Systems (MAS) through three interconnected tasks, each building upon the previous to deepen our understanding of agent-based intelligence.

The first task focused on implementing a multi-agent auction scenario, featuring one seller and multiple buyers. This exercise effectively demonstrated core MAS principles such as agent communication, negotiation protocols, and dynamic decision-making within a competitive context. It showcased how autonomous agents can interact, strategically raise offers, and adapt their behavior in response to evolving information and predefined negotiation rules.

Building on this foundation, the second task introduced the advanced dimensions of agent mobility and multi-criteria decision-making. By enabling a buyer agent to migrate across different containers and platforms to engage multiple sellers, we addressed challenges related to inter-container and inter-platform communication and platform configuration. This section highlighted the critical role of mobility in distributed MAS environments and incorporated weighted decision criteria to guide the buyer's optimal selection, reflecting real-world complexity and intelligent agent behavior.

The final part expanded our scope to advanced AI planning by experimenting with the Partial Order Planning (POP) algorithm from the AIMA project. This allowed us to engage with sophisticated planning techniques, including causal link management and constraint satisfaction, thereby enriching our grasp of how goal-driven agents formulate and execute plans autonomously.

Overall, this project bridged theory and practice by enabling us to design, implement, and evaluate multi-agent architectures capable of complex interactions and intelligent decision-making. The skills and insights gained here lay essential groundwork for future work in developing robust autonomous systems able to operate effectively in dynamic, distributed environments.